BOSTON REAL ESTATE

DATA SCIENCE REPORT

by Kadir Arslan

Created: 15.03.2025

Last Edited: 28.03.2026

Boston Real Estate Report

Welcome to 1970s Boston—a city shaped by economic shifts, urban renewal, and changing neighborhoods. In this report, we are going to take on the role of a real estate analyst to better understand what determines real estate prices and how various factors influence property values. Using historical data collected during this period, we are going to examine key characteristics such as place of residence, living conditions, and socioeconomic indicators to identify meaningful patterns.

Our goal is to go beyond mere observation and develop a deep understanding of the housing market by analyzing the relationships within the data and identifying the variables with the greatest influence. In doing so, we are going to analyze trends, highlight interesting findings, and examine how various characteristics contribute to price differences. This process will ultimately enable us to estimate property values with greater accuracy while also gaining insights into the dynamics of the Boston real estate market. In addition, we aim to translate these findings into practical insights that enable more informed decisions in the real estate sector.

Source Data Files (CSV Format)

Nobel Prize Data: github.com

Imported Libraries:

```
import pandas as pd
import numpy as np
import seaborn as sns
import plotly.express as px
import matplotlib.pyplot as plt
from pandas.plotting import register_matplotlib_converters
from sklearn.linear_model import LinearRegression
import matplotlib.dates as mdates
```

Display Formatting for Financial Data

To improve readability and ensure consistent formatting, we are going to configure pandas to display floating-point numbers and values in a more appropriate way.

```
# Create locators for ticks:
register_matplotlib_converters()


# Notebook Presentation:
pd.options.display.float_format = '{:,.2f}'.format
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
```

Acquiring the Source and Exploring Initial Dataset

```
data = pd.read_csv('csv_files/boston.csv', index_col=0)
```

The dataset was compiled in 1978 by researchers David Harrison and Daniel Rubinfeld using data from the 1970 U.S. Census covering 506 neighborhoods in the Greater Boston area. Their main goal was to measure the economic impact of air pollution by determining how much homebuyers were willing to pay for clean air.

To accomplish this, they built a model analyzing the relationship between median housing prices and 13 other variables, including local nitric oxide levels, crime rates, and pupil-to-teacher ratios.

Shape of DataFrame:

```
print("\nShape of DataFrame:")
print(data.shape)
```

```
(506, 14)
```

As indicated by the results, DataFrame has 506 entries (real estates in Boston) with 14 different columns (features) in each entry.

Features (Columns) of DataFrame:

```
print("\nColumns of DataFrame:")
print(data.columns)
```

```
Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
       'PTRATIO', 'B', 'LSTAT', 'PRICE'],
      dtype='str')
```

These attributes represent the key features of Boston's neighborhoods and properties and take into account factors such as living conditions, accessibility, environmental quality, and socioeconomic status.

Together, they will help us to analyze the real estate market and understand how various factors—particularly issues such as pollution and location—can influence property values and buyer preferences.

Meanings of Attributes:

1	**CRIM**	Crime rate per capita by location
2	**ZN**	Percentage of land reserved for properties larger than 25,000 square feet.
3	**INDUS**	Percentage of non-retail area per location.
4	**CHAS**	Charles River-Variable (= 1 if the area borders the river; otherwise, 0)
5	**NOX**	Nitrogen dioxide concentration (parts per 10 million)
6	**RM**	Average number of rooms per apartment
7	**AGE**	Percentage of owner-occupied homes built before 1940.
8	**DIS**	Weighted distances to five business centers in Boston.
9	**RAD**	Index of Accessibility of Ring Roads
10	**TAX**	Full-value Property tax rate per $10,000
11	**PTRATIO**	Pupil-Teacher ratio by location
12	**B**	Proportion of Black residents in a town(differs from a reference (0.63))
13	**LSTAT**	Percentage of people in a town from lower socioeconomic status.
14	**PRICE**	Median value of owner-occupied homes in $1000's

Initial Insight into the DataFrame:

To begin our analysis, we'll take a quick look at the dataset using the .head() function. This function displays the first few rows and helps us understand the structure and content of the data.

```
print("\nFirst Entries of DataFrame:")
print(data.head())
```

First Entries of DataFrame:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	PRICE
0	0.01	18.00	2.31	0.00	0.54	6.58	65.20	4.09	1.00	296.00	15.30	396.90	4.98	24.00
1	0.03	0.00	7.07	0.00	0.47	6.42	78.90	4.97	2.00	242.00	17.80	396.90	9.14	21.60
2	0.03	0.00	7.07	0.00	0.47	7.18	61.10	4.97	2.00	242.00	17.80	392.83	4.03	34.70
3	0.03	0.00	2.18	0.00	0.46	7.00	45.80	6.06	3.00	222.00	18.70	394.63	2.94	33.40
4	0.07	0.00	2.18	0.00	0.46	7.15	54.20	6.06	3.00	222.00	18.70	396.90	5.33	36.20

This initial preview shows that each row represents a different neighborhood in Boston, with several characteristics describing its features, as well as the corresponding real estate price. The dataset appears clean and well-structured, making it suitable for further analysis and modeling.

Note on prices: Since the data is from 1970, the PRICE values are given in thousands of dollars at current prices—so a value of 24.0 corresponds to $24,000, which was typical for real estate prices at that time.

Checking for Missing and Duplicated Values:

Before we continue, it is important to check the data quality by looking for missing or duplicate entries that could affect the analysis.

```
print("\nIs there any missing values?:")
print(data.isna().values.any())
print("\nIs there any duplicated values?:")
print(data.duplicated().values.any())
```

```
Is there any missing values?:
False
Is there any duplicated values?:
False
```

The results show that the dataset contains no missing or duplicate values, indicating that the data has been cleaned and is ready for further analysis and modeling.

Statistical Overview of the Dataset:

In this section, we examine the overall distribution and the key statistical measures of each variable using the .describe() function.

This provides a quick overview of the central tendencies, the dispersion, and the range of values within the dataset.

```
print("\nDescriptive Analysis of the DataFrame:")
print(data.describe())
```

Descriptive Analysis of the DataFrame:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	PRICE
count	506	506	506	506	506	506	506	506	506	506	506	506	506	506
mean	3.61	11.36	11.14	0.07	0.55	6.28	68.57	3.80	9.55	408.24	18.46	356.67	12.65	22.53
std	8.60	23.32	6.86	0.25	0.12	0.70	28.15	2.11	8.71	168.54	2.16	91.29	7.14	9.20
min	0.01	0.00	0.46	0.00	0.39	3.56	2.90	1.13	1.00	187.00	12.60	0.32	1.73	5.00
25%	0.08	0.00	5.19	0.00	0.45	5.89	45.02	2.10	4.00	279.00	17.40	375.38	6.95	17.02
50%	0.26	0.00	9.69	0.00	0.54	6.21	77.50	3.21	5.00	330.00	19.05	391.44	11.36	21.20
75%	3.68	12.50	18.10	0.00	0.62	6.62	94.07	5.19	24.00	666.00	20.20	396.23	16.96	25.00
max	88.98	100.00	27.74	1.00	0.87	8.78	100.00	12.13	24.00	711.00	22.00	396.90	37.97	50.00

The summary statistics reveal several useful insights about the dataset. On average, there are about 18.46 students per teacher, which suggests that classrooms in the various locations are moderately overcrowded.

The average property price is 22.53 (≈ $22,530), reflecting typical property values in Boston during the 1970s, while prices vary widely—from $5,000 to $50,000—indicating significant differences in property values across different neighborhoods.

Looking at housing characteristics, the average number of rooms per apartment is approximately 6.28, with apartments ranging from a minimum of 3.56 to a maximum of 8.78 rooms, indicating a mix of smaller and more spacious apartments.

Overall, the variation in values across various characteristics—such as crime rates, tax rates, and pollution levels—suggests that the Boston real estate market was quite diverse at that time, making it an interesting dataset for further analysis.

Note:

According to the U.S. Bureau of Labor Statistics, the cumulative inflation rate between 1970 and 2025 was approximately **740%.**

Therefore, the average home price of **$22,530** in 1970 would be nearly **$189,252** in 2025, giving a more realistic perspective on how housing values compare to modern markets.

Visualizing Key Distributions in the Dataset

After reviewing the summary statistics, the data becomes much easier to understand once we visualize how the values are distributed across the various characteristics.

In this section, we are going to create histogram charts in combination with KDE (Kernel Density Estimation) plots to identify patterns, detect skewness, and pinpoint potential outliers in key variables.

These visualizations help us go beyond mere statistics and develop a more intuitive understanding of how factors such as price, accessibility, and housing features interact.

Organizing Plot Configurations Efficiently

To make the code more readable and avoid repetition, we store the display parameters for each variable in a single, dictionary-like configuration.

This approach allows us to easily iterate through multiple variables and create consistent, well-formatted visualizations with minimal code duplication.

```python
#defining the arrangements for each plot in one dictionary to optimize the code.
plot_configs = [
    {
        'column': 'PRICE',
        'color': '#00B8A9',  # Modern soft blue
        'bins': 50,
        'title': 'House Prices in Boston',
        'avg_text': 'Average: ${:,.2f}',
        'multiplier': 1000,  # Used to scale the average calculation
        'xlabel': 'Price (in $1000)',
        'ylabel': 'Number of Homes'
    },
    {
        'column': 'DIS',
        'color': '#4E9F3D',  # Modern soft green
        'bins': 50,
        'title': 'Distance to Employment Centres',
        'avg_text': 'Average Distance: {:.2f}',
        'multiplier': 1,
        'xlabel': 'Weighted Distance to 5 Boston Employment Centres',
        'ylabel': 'Number of Homes'
    },
    {
        'column': 'RM',
        'color': '#FF9A00',  # Modern soft purple
        'bins': 50,
        'title': 'Distribution of Rooms in Boston',
        'avg_text': 'Average Rooms: {:.2f}',
        'multiplier': 1,
        'xlabel': 'Average Number of Rooms per Owner Unit',
        'ylabel': 'Number of Homes'
    },
    {
        'column': 'RAD',
        'color': '#F6416C',  # Modern soft red
        'bins': 20,
        'title': 'Accessibility to Highways in Boston',
        'avg_text': 'Average Index: {:.2f}',
        'multiplier': 1,
        'xlabel': 'Index of Accessibility to Highways',
        'ylabel': 'Number of Houses'
    }
]
```

Generating Histogram and KDE Plots

Using the predefined configurations, we iterate through each variable and generate histogram charts, supplemented with KDE curves to better illustrate the underlying distribution.

Additionally, we include average values directly in the titles, making each visualization more informative and easier to interpret at a glance.

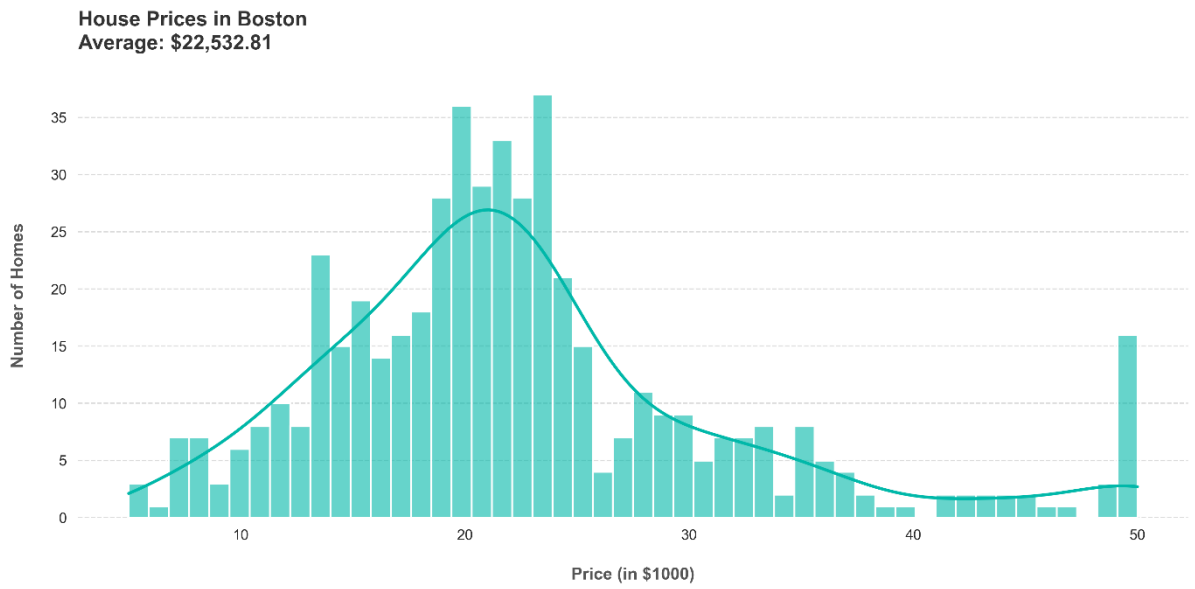
code:

```
# Firstly, we create a general theme for thr Graphs
sns.set_theme(style="whitegrid", font_scale=1.1)
plt.rcParams['font.family'] = 'sans-serif'

# Here we are going to create a graph for each value in dictionary.
for plot in plot_configs:
    plt.figure(figsize=(14, 7), dpi=300)

    # we display some values on the graph:
    avg_val = data[plot['column']].mean() * plot['multiplier']
    formatted_avg = plot['avg_text'].format(avg_val)
    full_title = f"{plot['title']}\n{formatted_avg}"

    # we are going to use histplot with kde
    ax = sns.histplot(
        data=data,
        x=plot['column'],
        bins=plot['bins'],
        kde=True,
        color=plot['color'],
        alpha=0.6,
        edgecolor='white',  # a subtle edge for the bars
        linewidth=1.5,
        line_kws={'linewidth': 2.5}
    )

    # Titles:
    plt.title(full_title, fontsize=17, fontweight='bold', pad=20, loc='left',
color='#333333')
    plt.xlabel(plot['xlabel'], fontsize=14, fontweight='bold', color='#555555',
labelpad=20)
    plt.ylabel(plot['ylabel'], fontsize=14, fontweight='bold', color='#555555',
labelpad=20)

    # Cleaning the gridlines:
    ax.grid(axis='y', linestyle='--', alpha=0.7)
```

```
    ax.grid(axis='x', visible=False)

    sns.despine(left=True, bottom=True)  # Removes the borders
    plt.tight_layout()
    plt.show()
```

Graphs:

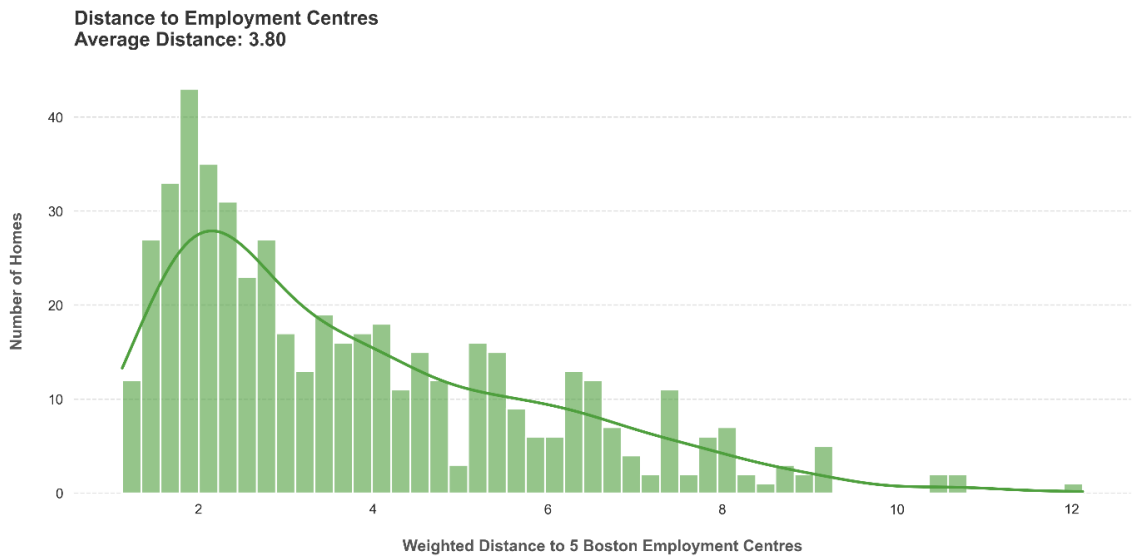
Distribution of House Prices (PRICE)



The distribution of real estate prices appears to be slightly skewed to the right, with most properties concentrated in the range between $15,000 and $25,000, while a smaller number of properties reach higher values close to $50,000.

This suggests that while most neighborhoods were affordable, there were a few premium locations, which may be due to more attractive settings or a cleaner environment with lower environmental impact.

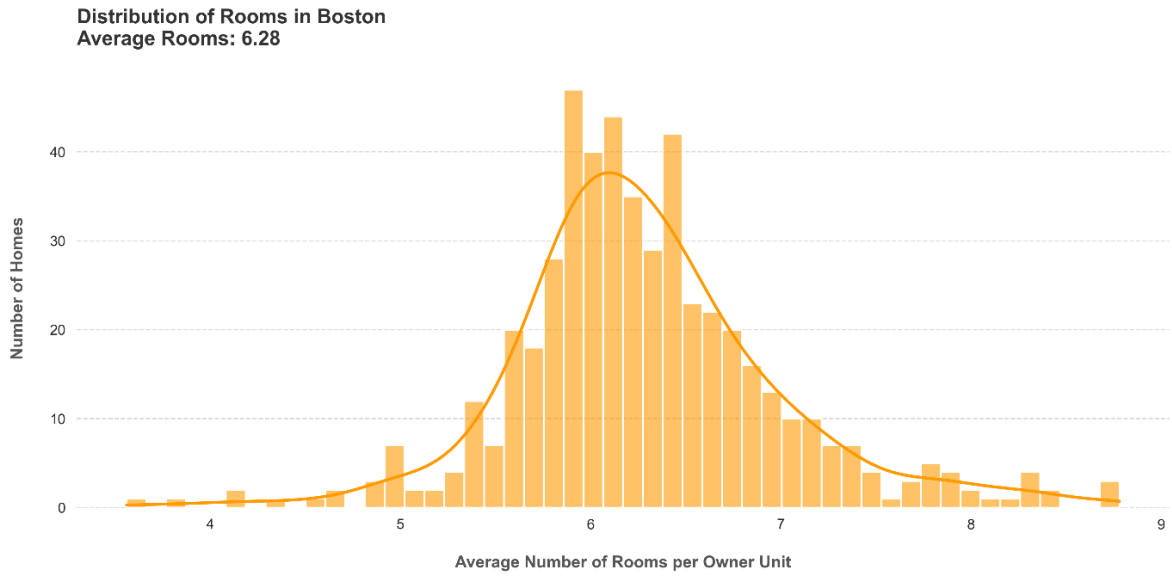
Distribution of Distance to Employment Centres (DIS)



The distribution of distances is clearly skewed to the right, with most apartments located relatively close to workplaces and fewer properties situated further away.

This suggests that proximity to workplaces was a key factor in urban planning and that housing located near workplaces was more common.

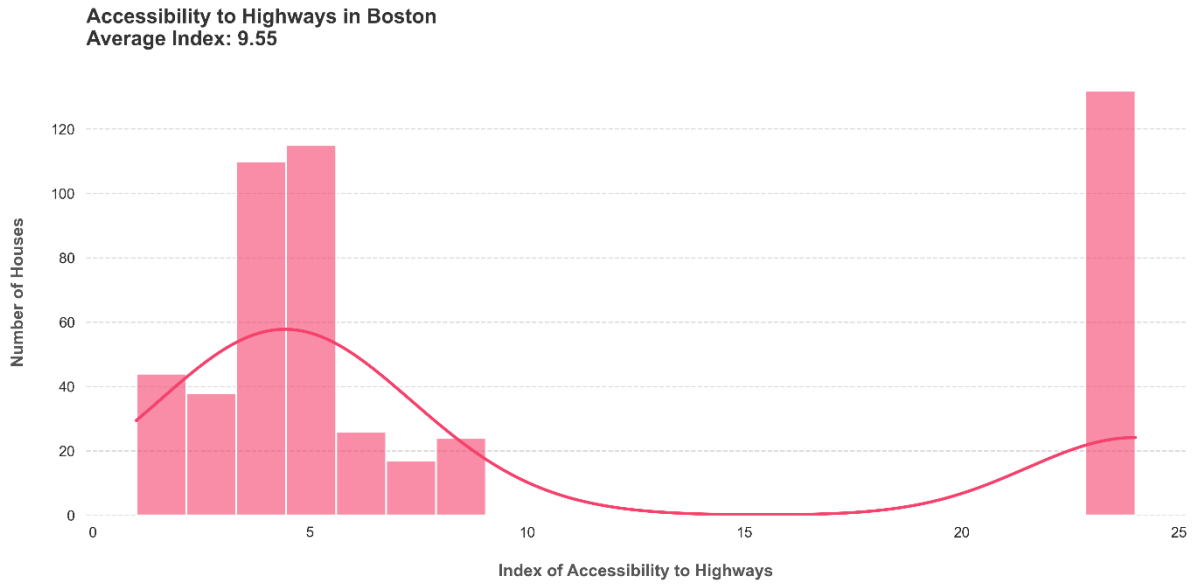
Distribution of Number of Rooms (RM)



The number of rooms follows a curve that is approximately normally distributed and has a mean of 6 rooms, suggesting that most houses were medium-sized and fairly uniform in design.

This points to a relatively standardized architectural style in Boston at that time, with only a few extremely small or very large houses.

Distribution of Accessibility to Highways (RAD)



The accessibility index exhibits a highly skewed distribution, with a high concentration of housing units in the lower range of values and a sharp increase at the upper end.

This pattern suggests that while most areas had moderate connectivity to the highway network, a certain group of locations exhibited exceptionally high connectivity, which is likely attributable to large urban centers or transportation hubs.

Number of Properties Near the Charles River

In this section, we will examine how many properties are located near the Charles River.

```
river_data = data.groupby('CHAS', as_index=False).agg({'CHAS': pd.Series.count})

print(river_data)
```

```
values   CHAS
    0    471
    1     35
```

The results show that only 35 properties are located along the river, while 471 properties are not located along the river.

This suggests that riverside properties are relatively rare in the dataset, which may make them more sought-after and valuable due to their limited availability and impressive appeal.

Exploring Feature Relationships Through Pairwise Visualizations

In this section, we are going to examine the relationships between different variables in the dataset by comparing them visually. Understanding these relationships helps us identify patterns, connections, and potential influences between factors such as environmental pollution, accessibility, and real estate prices.

Before creating a model, it is important to develop an understanding of how variables might interact with one another—for example, whether cleaner areas are located farther from the city center or whether larger homes tend to command higher prices.

Creating the Pair Plots

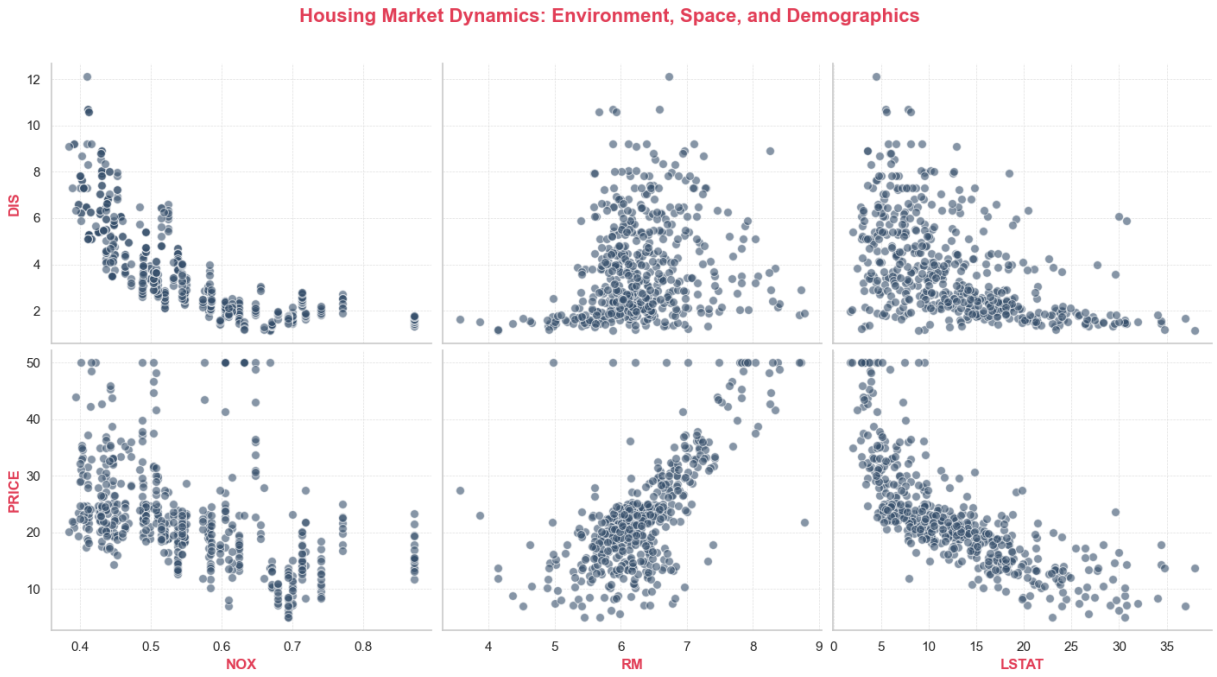
To illustrate these relationships, we use Seaborn's $.pairplot()$ function, which allows us to display all pairwise interactions in a single, clear grid.

This tool not only displays scatter plots between variables, but also illustrates the distribution of individual characteristics, making it easier to identify correlations, trends, and unusual patterns in the data.

code:

```
sns.set_theme(style="whitegrid", palette="mako")

plt.figure(figsize=(18, 6), dpi=300)

g = sns.pairplot(
    data,
    x_vars=["NOX", "RM", "LSTAT"],
    y_vars=["DIS", "PRICE"],
    kind='scatter',
    height=4,
    aspect=1.2, # width-to-height ratio
    plot_kws={
        'alpha': 0.6,  # Transparency for the dots
        's': 50,  # point sizes
        'edgecolor': 'white',  # A subtle white border around dots
        'linewidth': 0.75,
        'color': '#364F6B'
    }
)

# Enhancing the layout
g.fig.suptitle("Housing Market Dynamics: Environment, Space, and Demographics",
               fontsize=16, fontweight='bold', color='#E23E57')

# labels:
for ax in g.axes.flatten():
    ax.set_xlabel(ax.get_xlabel(), fontweight='bold', color='#E23E57')
    ax.set_ylabel(ax.get_ylabel(), fontweight='bold', color='#E23E57')
    ax.grid(color='#e0e0e0', linestyle='--', linewidth=0.5)

g.fig.subplots_adjust(top=0.90)
plt.show()
```

Graph:



The scatter plot reveals several strong and intuitive correlations between key variables in the real estate market and, in particular, illustrates how environmental and socioeconomic factors influence property values.

We observe a clear negative correlation between air pollution (NOX) and distance (DIS), suggesting that cleaner areas tend to be located farther from industrial centers, highlighting the historical urban development patterns in Boston in the 1970s.

In addition, there is a strong positive correlation between the number of rooms (RM) and property prices (PRICE), suggesting that larger apartments consistently have a higher value. In contrast, the LSTAT score (low socioeconomic status) shows a strong negative correlation with price, suggesting that areas with higher poverty rates tend to have significantly lower property values, reflecting the broader economic and social inequalities observed in real estate markets.

Interestingly, the relationship between NOX (air pollution) and PRICE appears to be weak and nonlinear, suggesting that air pollution alone does not directly determine real estate prices. This could be because this effect is indirect and intertwined with other factors—for example, heavily polluted areas may still command higher prices if they are closer to workplaces or offer better accessibility, while cleaner areas may be farther away and less accessible.

Deep Dive into Feature Relationships with Joint Plot

While scatter plots provide us with a general overview of the relationships, this section focuses on examining specific pairs of variables in greater detail.

By identifying key relationships—such as the relationship between pollution and distance, or between the number of rooms and price—we can better understand the strength, direction, and structure of these interactions

This allows us to identify more nuanced patterns that might not be immediately apparent in larger, aggregated visualizations.

Structuring Relationship Visualizations

To streamline the process and avoid duplicate code, we define all variable pairs and their visual settings within a single configuration structure.

This approach improves readability and makes it easier to manage and scale our visual analysis.

```
# Creating a list of dictionaries that we are going the plot at the graph:
plots_relations = [
    {'x': 'DIS', 'y': 'NOX', 'color': '#7f3c8d', 'title': 'Distance to Employment
vs. Nitric Oxides'},
    {'x': 'INDUS', 'y': 'NOX', 'color': '#11a579', 'title': 'Non-Retail Business
Acres vs. Nitric Oxides'},
    {'x': 'LSTAT', 'y': 'RM', 'color': '#3969ac', 'title': 'Lower Status
Population vs. Number of Rooms'},
    {'x': 'LSTAT', 'y': 'PRICE', 'color': '#f2b701', 'title': 'Lower Status
Population vs. Home Price'},
    {'x': 'RM', 'y': 'PRICE', 'color': '#e73f74', 'title': 'Number of Rooms vs.
Home Price'}
]
```

Creating Joint Plots for Detailed Analysis:

Using Seaborn’s `jointplot`, we create focused scatter plots for each relationship, supplemented by marginal distributions, to better understand the behavior of the variables.

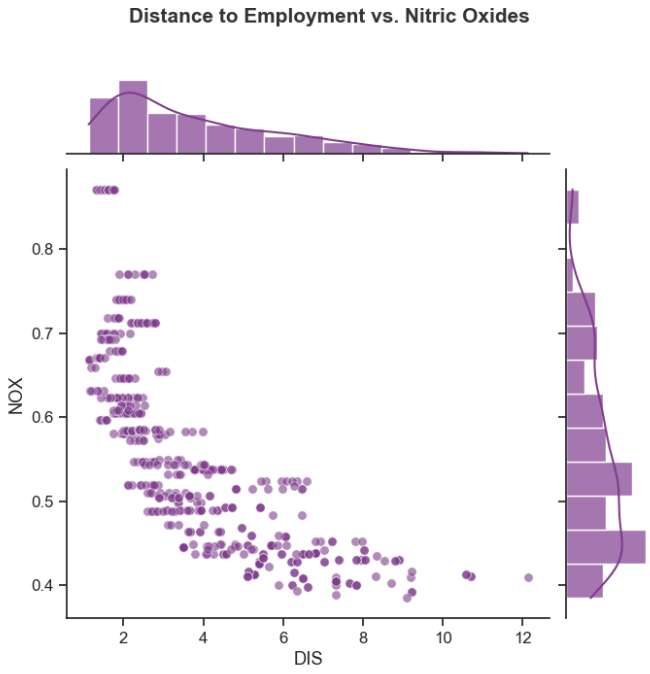
By adjusting the transparency (alpha), overlapping data points become easier to spot, making patterns and densities more clearly visible and easier to interpret.

code:

```
# Setting a clean theme
sns.set_theme(style="ticks", context="notebook", font_scale=1.1)
plt.figure(figsize=(18, 6), dpi=300)

# Looping through the plots and genereting jointplots for every relation
for plot in plots_relations:
    g = sns.jointplot(
        data=data,
        x=plot['x'],
        y=plot['y'],
        kind='scatter',
        height=7,
        color=plot['color'],
        joint_kws={
            'alpha': 0.6,  # Transparency for the density
            's': 45,  # not too big dot size
            'edgecolor': 'white',  # little edge for the borders around dots
            'linewidth': 0.5
        },
        # arrangements for the histograms
        marginal_kws={
            'kde': True,  # plotting a smooth density curve over the bars
            'alpha': 0.7  # a subtle transparency for the bars
        }
    )

    # title and label allignments:
    g.fig.suptitle(plot['title'], y=1.0, fontsize=15, fontweight='bold',
color='#333333')

    g.fig.subplots_adjust(top=0.90)


    plt.show()
```

Graphs:

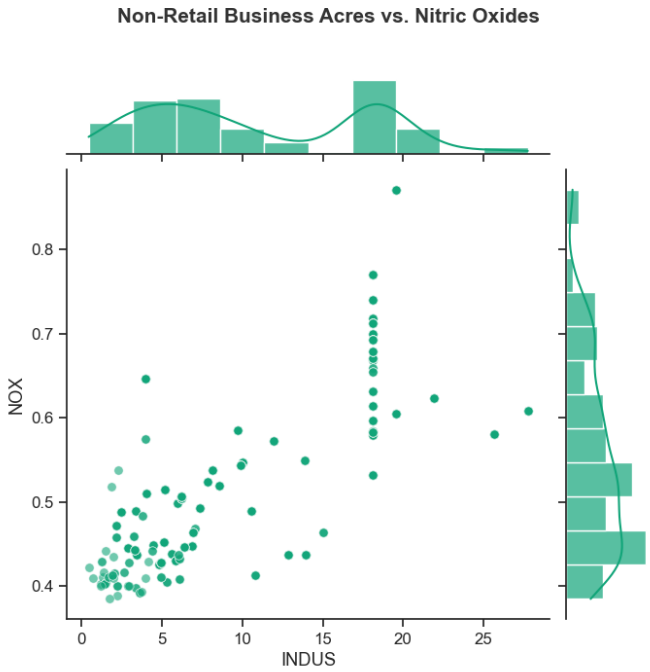
Distance to Employment vs. Nitric Oxides (DIS vs NOX)



There is a strong negative correlation: pollutant levels decrease with increasing distance from workplaces, suggesting that industrial areas and city centers are the main sources of nitrogen oxide emissions.

This reflects the actual situation, as downtown Boston was more industrialized in the 1970s, meaning that people who lived further out likely had cleaner air but may have had to travel longer distances to work.

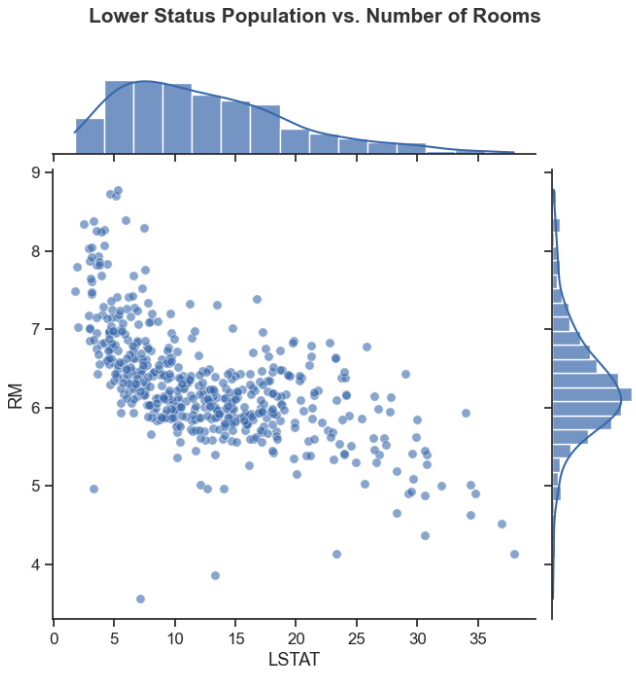
Non-Retail Business Areas vs. Nitric Oxides (INDUS vs NOX)



There is a clear positive correlation suggesting that areas with more industrial land tend to have higher levels of environmental pollution.

This is to be expected, as non-retail areas often house factories and manufacturing facilities, which supports the assumption that industrial activity has significantly contributed to the environmental conditions that affect residential construction.

Lower Status Population vs. Number of Rooms (LSTAT vs RM)



As mentioned earlier, there is a strong negative correlation: higher LSTAT scores are associated with lower real estate prices.

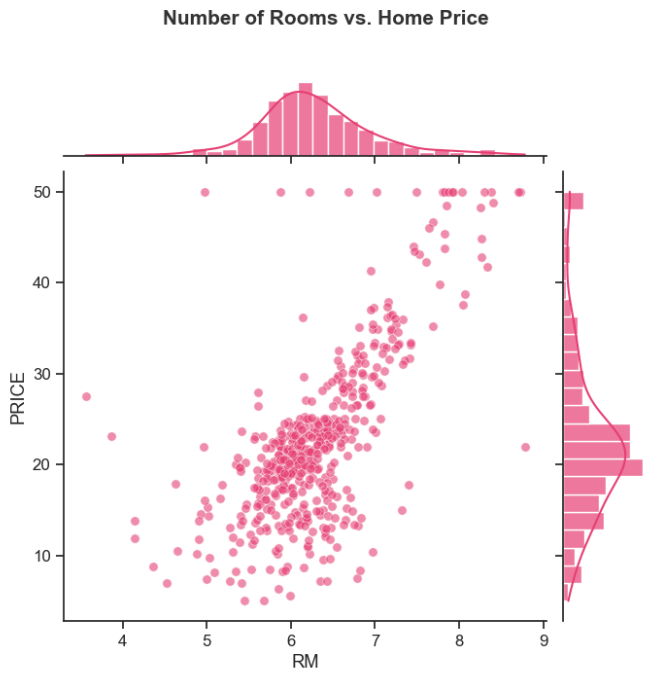
This suggests that socioeconomic conditions are among the most influential factors in determining real estate values.

Lower Status Population vs. Home Price (LSTAT vs PRICE)

As observed earlier, there is a strong negative correlation: higher LSTAT scores are associated with lower real estate prices.

This suggests that socioeconomic conditions are among the most influential factors in determining real estate values.

Number of Rooms vs. Home Price (RM vs PRICE)



Consistent with previous observations, a clear positive correlation can be observed: larger homes generally fetch higher prices.

This supports the assumption that the size of a property is a key factor in determining its value on the real estate market.

Building a Predictive Framework: Train-Test Split Strategy

In this section, we are going to begin preparing our data for predictive modeling by applying a statistical estimation approach.

Instead of using the entire dataset for training, we split it into separate subsets to ensure that our model can be evaluated using previously unseen data.

This approach helps simulate real-world scenarios in which the model must make predictions based on new inputs, allowing us to better assess its reliability and performance.

Importing the Train-Test Split Tool

We import the $train\_test\_split$ function from Scikit-learn, which allows us to efficiently split our dataset into training and test subsets.

```
from sklearn.model_selection import train_test_split
```

Defining Target and Feature Variables

Here, we divide the dataset into features (independent variables) and the target variable (dependent variable), which we want to predict.

The “PRICE” column is defined as our target variable, while all other variables serve as input data for estimating real estate prices.

```
target_variable = data['PRICE']
features = data.drop('PRICE', axis=1)
```

Splitting the Dataset

We split the data into training and test sets in an 80:20 ratio, with 80% of the data used to train the model and 20% reserved for evaluation.

Using a fixed $random\_state$ ensures that the distribution remains consistent across all runs, making the results reproducible and easier to compare.

```
X_train, X_test, y_train, y_test = train_test_split(features,
                                                    target_variable,
                                                    test_size=0.2,
                                                    random_state=10)
```

Evaluating the Data Split Distribution:

```
# % of training set
train_pct = len(X_train)/len(features) * 100
print(f'Training data is {train_pct:.3}% of the total data.')


# % of test data set
test_pct = 100 * X_test.shape[0] / features.shape[0]
print(f'Test data makes up the remaining {test_pct:0.3}%.')
```

```
Training data is 79.8% of the total data.
Test data makes up the remaining 20.2%.
```

The results show that approximately 79.8% of the data is used for training, while 20.2% is reserved for testing, which is a common and balanced approach in machine learning.

This means that the model learns patterns from the majority of the data while being tested on a significant portion of the data that it has never seen before.

This approach helps us reduce the risk of overfitting and gives us a more realistic picture of how well our model might perform when applied to new, real-world housing data.

Modeling Housing Prices with Multivariable Linear Regression

In this section, we are going to build a predictive model using multivariate linear regression to estimate real estate prices based on multiple features simultaneously.

Since our dataset contains 13 different variables, the model learns how each factor—such as pollution, number of rooms, or socioeconomic conditions—affects the final price. This approach allows us to go beyond simple correlations and quantify the combined impact of all variables on property values.

Ultimately, the goal is to develop a model that generalizes well and provides reliable price estimates for previously unknown data.

Initializing the Linear Regression Model

We import and initialize the $LinearRegression$ model from Scikit-learn, which provides a simple yet powerful way to model linear relationships between multiple features and a target variable.

This model estimates coefficients for each variable, thereby helping us understand their individual impact on real estate prices.

```
from sklearn.linear_model import LinearRegression
Regression = LinearRegression()
```

Training the Regression Model

We train the regression model using the training dataset so that it can learn the relationships between the input features and the target variable (PRICE).

```
Regression.fit(X_train,y_train)
```

Evaluating Model Performance with R^2 Score

```
# Coefficient of Determination
r_squared = Regression.score(X_train, y_train)
print(f'Training data r-squared: {r_squared:.2}')
```

```
Training data r-squared: 0.75
```

The R-squared value (coefficient of determination) indicates how well the model explains fluctuations in real estate prices based on the input variables. An R^2 value of 0.75 means that the model can explain about 75% of the variation in real estate prices, which is considered quite meaningful for real-world data.

This suggests that the selected variables capture a significant portion of the factors influencing real estate prices, although 25% of the variance remains unexplained—likely due to factors not included in the dataset or to randomness inherent in the market.

Overall, this is a solid result, suggesting that the model has identified meaningful patterns and has good predictive power without being too perfect (which could indicate overfitting).

Interpreting Model Coefficients: Understanding Feature Impact

In this section, we examine the regression coefficients to understand how individual characteristics affect real estate prices. These coefficients indicate the expected change in price when the respective variable changes by one unit, assuming that all other variables remain constant.

This step is essential for verifying whether the model's behavior meets expectations in practice.

The model includes 13 explanatory variables, and therefore the multivariable linear regression can be expressed in the following form:

$$\text{PRICE} = \theta_0 + \theta_1 \cdot \text{CRIM} + \theta_2 \cdot \text{NOX} + \theta_3 \cdot \text{DIS} + \theta_4 \cdot \text{CHAS} + \cdots + \theta_{13} \cdot \text{LSTAT}$$

We are going to convert the coefficients into a new DataFrame for a better readability.

```
regression_coefficients = pd.DataFrame(data=Regression.coef_,
index=X_train.columns, columns=['Coefficient'])

print(regression_coefficients)
```

```
         Coefficient
CRIM           -0.13
ZN              0.06
INDUS          -0.01
CHAS            1.97
NOX           -16.27
RM              3.11
AGE             0.02
DIS            -1.48
RAD             0.30
TAX            -0.01
PTRATIO        -0.82
B               0.01
LSTAT          -0.58
```

The coefficients generally follow logical and understandable patterns, which is a clear indication that the model captures meaningful relationships.

RM (number of rooms) has a strongly positive coefficient, confirming that larger apartments tend to have a higher value, while LSTAT (lower socioeconomic status) has a negative effect, suggesting that higher poverty levels reduce property values. Furthermore, NOX (air pollution) has a high negative coefficient, suggesting that air quality plays a significant role in pricing—which reinforces the original rationale behind the dataset regarding the value of clean air.

However, some coefficients—such as INDUS or AGE—show only very small effects, suggesting that, when other variables are taken into account, they may not be strong individual predictors. Overall, the signs and magnitudes of the coefficients are consistent with economic intuition, which strengthens confidence that the model is both plausible and interpretable.

Estimating the Value of an Additional Room

In this section, we take a closer look at how a specific characteristic—the number of rooms (Z)—affects real estate prices.

By analyzing the regression coefficient, we can estimate the monetary value that each additional room adds to a property.

```
# # Premium for having an extra room
extra = regression_coefficients.loc['RM'].values[0] * 1000  # i.e., ~3.11 * 1000
print(f'The additional price for having an extra room is ${extra:.5}')
```

```
The price premium for having an extra room is $3108.5
```

According to the model, adding an extra room increases the price of a house by about $3,108 (in 1970 prices).

Adjusted for inflation (~740%), this amounts to roughly $26,107 today, which illustrates just how much a property's size influences its value and reflects a consistent trend in practice: larger living spaces command higher prices.

Evaluating Model Performance Through Predictions and Residuals

In this section, we take a closer look at how well our regression model performs by analyzing its predictions and errors. While the R^2 value gives us a general idea of the model's fit, it does not reveal where or how the model makes mistakes.

By analyzing the residuals which are the differences between the actual and predicted values, we gain a clearer picture of the model's accuracy and its potential weaknesses.

Calculating Residuals

First, we use the trained model to generate predicted values, and then we calculate the residuals by subtracting these predictions from the actual values.

These residuals represent the model's errors and are essential for evaluating its performance and identifying patterns that the model may have overlooked.

```
predicted_values = Regression.predict(X_train)
residuals = (y_train - predicted_values)
```

Crating the Scatter Graph:

To better assess the model's performance, we create two scatter plots: one comparing the actual values with the predicted values, and another comparing the residuals with the predicted prices.

code:

```
sns.set_theme(style="whitegrid", palette="deep")

# we are going to create a graph with 1 row and 2 columns.
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(14, 6), dpi=300)

# First Plot : Actual vs Predicted values
sns.scatterplot(x=y_train, y=predicted_values, ax=axes[0], color='#2c3e50',
alpha=0.6, s=60, edgecolor=None)

# The ideal prediction line (x=y) (y_train = y_train).
axes[0].plot(y_train, y_train, color='#F45B26', linewidth=2, linestyle='--')

axes[0].set_title('Actual vs. Predicted Prices', fontsize=15, fontweight='bold')
axes[0].set_xlabel('Actual Prices in $1000', fontsize=12)
axes[0].set_ylabel('Predicted Prices in $1000', fontsize=12)

# Second plot: Distribution of residuals
sns.scatterplot(x=predicted_values, y=residuals, ax=axes[1], color='#406AAF',
alpha=0.6, s=60, edgecolor=None)

# reference for residual points
axes[1].axhline(y=0, color='#F45B26', linestyle='--', linewidth=2, alpha=0.8)

axes[1].set_title('Residuals vs. Predicted Values', fontsize=15)
axes[1].set_xlabel('Predicted Prices', fontsize=12)
axes[1].set_ylabel('Residuals', fontsize=12)

fig.suptitle('Regression Model Performance', fontsize=20, color='#F45B26',
fontweight='bold', y=1.0)

# Layout arrangements and print
sns.despine(left=True, bottom=True)
plt.tight_layout()
plt.show()
```

Graphs:

The “Actual vs. Predicted Prices” chart shows that the data points closely align with the reference line, suggesting that the model performs quite well overall, even though it tends to underestimate more expensive properties and slightly overestimate less expensive ones.

This pattern is common in linear models and suggests that outliers (particularly in the case of luxury real estate) may not be fully captured, as nonlinear relationships or unobserved factors—such as the reputation of the neighborhood or the infrastructure—are not taken into account.

In the “Residuals vs. Predicted Residuals” plot, the residuals are mostly clustered around zero, which is a good sign; however, at higher forecast values, a slight increase in dispersion can be observed, suggesting unequal variance.

This suggests that forecasting errors increase for more expensive properties, likely because prices in the premium segment are influenced by more complex and less quantifiable factors, making it difficult to produce an accurate estimate using a simple linear model.

Assessing Residual Distribution: Mean and Skewness Analysis

In this section, we further evaluate our model by analyzing the statistical properties of the residuals. Since residuals represent the model's prediction errors, examining their distribution helps us determine whether the model is unbiased and performs well.

Ideally, the residuals should follow a normal distribution with a mean of zero, which suggests that the errors are random rather than systematic.

Calculating the Mean and the Skew:

```
mean_residual = round(residuals.mean(), 2)
skew_residual = round(residuals.skew(), 2)
```

A mean close to 0 indicates that the model neither consistently overestimates nor underestimates prices, while a skewness close to 0 suggests a symmetric error distribution.

If the skewness deviates significantly from zero, this may indicate that the model performs differently across certain value ranges, for example, by systematically undervaluing expensive properties.

Overall, this analysis provides a more nuanced understanding of model performance that goes beyond simple accuracy metrics.

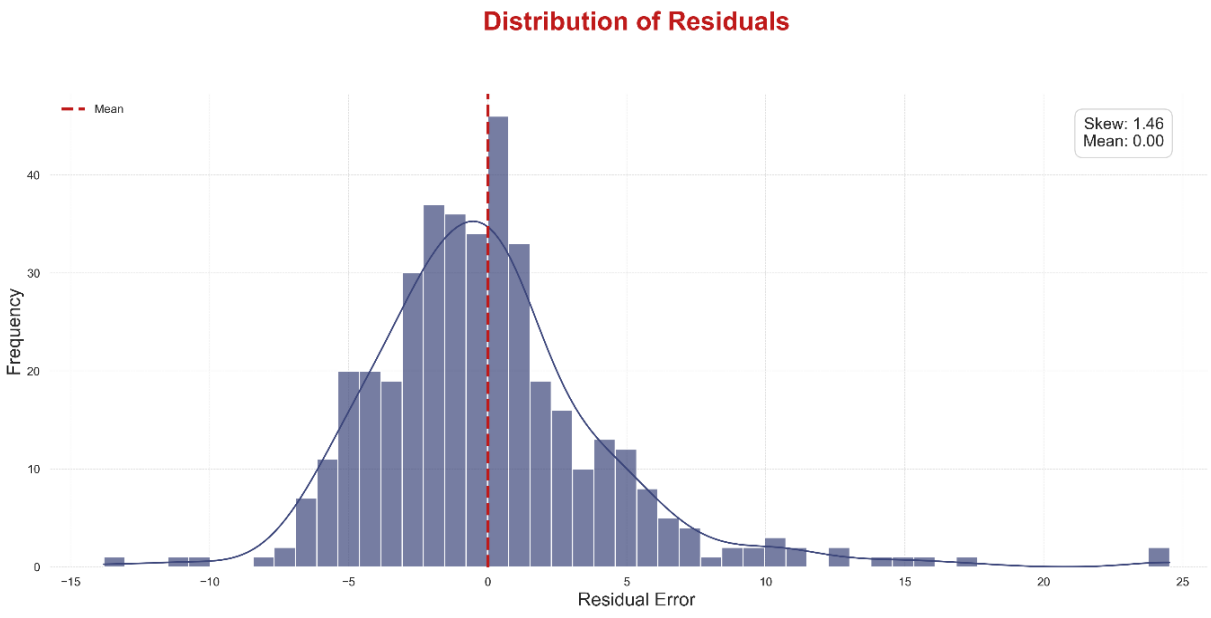
Creating the Histogram Plot:

code:

```
plt.figure(figsize=(16, 8), dpi=300)
sns.set_theme(style="whitegrid")

ax = sns.histplot(
    residuals,
    bins=50,
    kde=True,
    color='#3C467B',
    edgecolor='white', # Adds crisp white borders between the bars
    linewidth=1,
    alpha=0.7 # Softens the color
)
```

```
# Adding a vertical line to display the mean:
plt.axvline(x=mean_residual, color='#BF1A1A', linestyle='--', linewidth=3,
label='Mean')

# Displaying mean and skew values:
stats_text = f"Skew: {skew_residual:.2f}\nMean: {mean_residual:.2f}"
plt.text(
    0.95, 0.95, stats_text,
    transform=ax.transAxes,
    fontsize=15, verticalalignment='top', horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='white', alpha=0.9))

# axes-labels:
plt.title('Distribution of Residuals', fontsize=24, fontweight='bold', pad=15,
color='#BF1A1A', y=1.1)
plt.xlabel('Residual Error', fontsize=14)
plt.ylabel('Frequency', fontsize=14)

ax.grid(color='#e0e0e0', linestyle='--', linewidth=0.5)
sns.despine(left=True, bottom=True)
plt.legend(frameon=False)
plt.tight_layout()

plt.show()
```

Graph:



The residual distribution is very close to zero (mean ≈ 0.00), which suggests that the model is generally unbiased and does not systematically overestimate or underestimate real estate prices.

However, the positive skewness (≈ 1.46) suggests that there are more extreme positive errors, meaning that the model tends to underestimate the value of more expensive properties, which is consistent with our observations in earlier charts.

This pattern is quite common in real estate modeling, as high-value properties are often influenced by complex, less quantifiable factors—such as the prestige of the location, architectural uniqueness, or proximity to sought-after amenities—that are not fully captured by the dataset.

Improving Model Fit with Data Transformation

In this section, we are going to aim to improve the performance of our model by accounting for the skewness observed in the data.

Since linear regression works best when the variables follow a normal (symmetric) distribution, highly skewed data can compromise the model's accuracy.

To improve this, we are considering applying a logarithmic transformation to the target variable (PRICE) in order to make its distribution more balanced and better suited for modeling.

Applying Log Transformation to the Target Variable:

We calculate the skewness of the original price distribution and then apply a logarithmic transformation using the NumPy function $.log()$.

By comparing the skewness before and after the transformation, we can determine whether the logarithmically transformed data is closer to a normal distribution.

```
# Transforming the variables
skew_price = data['PRICE'].skew()
log_price = np.log(data['PRICE'])
skew_log_price = log_price.skew()
```

Creating Dual Histogram Plots:

To visually assess the improvement, we create two histograms: one for the original prices and one for the logarithmically transformed prices.

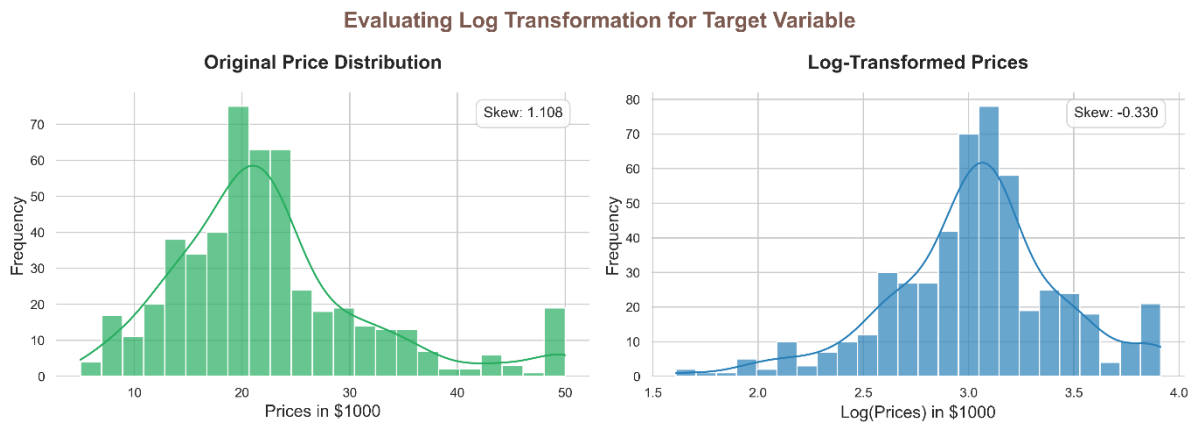
This direct comparison helps us clearly see whether the transformation reduces the skewness and produces a more symmetrical distribution, which is generally better suited for linear regression models.

code:

```
sns.set_theme(style="whitegrid")

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(14, 5), dpi=300)

# First Graph: Original price graph:
sns.histplot(
    data['PRICE'],
    kde=True,
    ax=axes[0],
    color='#27ae60', edgecolor='white',
    linewidth=1, alpha=0.7
)

axes[0].text(
    0.95, 0.95, f"Skew: {skew_price:.3f}",
    transform=axes[0].transAxes,
    fontsize=12, verticalalignment='top', horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='white', alpha=0.9))

axes[0].set_title('Original Price Distribution', fontsize=16, fontweight='bold')
axes[0].set_xlabel('Prices', fontsize=14)
axes[0].set_ylabel('Frequency', fontsize=14)


# Second Graph:
sns.histplot(
    log_price,
    kde=True,
    ax=axes[1],
    color='#2980b9', edgecolor='white',
    linewidth=1, alpha=0.7
)

axes[1].text(
    0.95, 0.95, f"Skew: {skew_log_price:.3f}",
    transform=axes[1].transAxes,
    fontsize=12, verticalalignment='top', horizontalalignment='right',
```

```
    bbox=dict(boxstyle='round,pad=0.5', facecolor='white', alpha=0.9))

axes[1].set_title('Log-Transformed Prices', fontsize=16, fontweight='bold')
axes[1].set_xlabel('Log(Prices)', fontsize=14)
axes[1].set_ylabel('Frequency', fontsize=14)

fig.suptitle('Evaluating Log Transformation for Target Variable', fontsize=18,
fontweight='bold', color='#7D5A50')
sns.despine(left=True, bottom=True)

plt.tight_layout()
plt.show()
```

Graph:



The original price distribution is clearly right-skewed (skewness ≈ 1.11), suggesting that while most homes are moderately priced, there are a few significantly more expensive properties that shift the distribution to the right.

After the log transformation, the distribution becomes significantly more symmetric (skewness ≈ -0.33), which suggests a better approximation to a normal distribution and makes it more suitable for linear regression models.

This improvement is due to the fact that the log transformation compresses extreme values, particularly prices in the upper segment, which are often influenced by specific factors such as prime locations or luxurious amenities—thereby causing the model to treat these values proportionally rather than as outliers.

Regression using Log Prices

In this section, we are going to refine the regression model by applying a logarithmic transformation to the dependent variable (PRICE).

Since we previously observed skewness in the price distribution, transforming the data helps stabilize the variance and improve the model's ability to capture the underlying relationships.

By using the same configuration as before to split the data into training and test sets, we ensure that any performance improvements can be clearly attributed to the transformation and not to changes in how the data is split.

The updated regression model:

$$\log(\text{PRICE}) = \theta_0 + \theta_1 \cdot \text{CRIM} + \theta_2 \cdot \text{NOX} + \theta_3 \cdot \text{DIS} + \theta_4 \cdot \text{CHAS} + \cdots + \theta_{13} \cdot \text{LSTAT}$$

Preparing the Log-Transformed Dataset:

Here, we redefine our target variable by using the natural logarithm of the prices, while leaving the feature set unchanged. This allows the model to learn proportional (percentage) relationships rather than absolute price changes.

```
new_target_variable = np.log(data['PRICE']) # Use log prices
features = data.drop('PRICE', axis=1)

X_train, X_test, log_y_train, log_y_test = train_test_split(features,
                                                            new_target_variable,
                                                            test_size=0.2,
                                                            random_state=10)
```

Training the Log-Linear Regression Model

We fit a new linear regression model using the logarithmically transformed response variable, which allows the model to better handle skewed distributions and outliers.

```
log_regression = LinearRegression()
log_regression.fit(X_train, log_y_train)
```

Calculating the new R-squared value (Coefficient of Determination)

```
log_r_squared = log_regression.score(X_train, log_y_train)
print(f'Training data r-squared:')
print(f'{log_r_squared:.2}')
```

```
Training data r-squared:
0.79
```

The new model achieves an R^2 value of 0.79, compared to 0.75 for the original model, indicating a significant improvement in explanatory power.

This suggests that the log transformation has made the relationships between the features and the target variable more linear, enabling the model to better capture fluctuations in real estate prices.

Interpreting Coefficients in the Log-Linear Model

In this section, we examine how the relationships between the variables and housing prices change after a logarithmic transformation. Even though the magnitude of the coefficient's changes, their sign (positive or negative) remains the key factor for interpretation, as it indicates whether a variable increases or decreases the value of the property.

This analysis helps us verify whether the model meets expectations in practice and provides more stable, proportional insights into price changes.

Examining Coefficient Values

We extract and organize the coefficients from the logarithmically transformed regression model to better understand the impact of individual features.

Compared to the original model, these coefficients now represent percentage price changes rather than absolute dollar amounts, making them easier to interpret in relative terms.

```
coefficients_log_regression = pd.DataFrame(data=log_regression.coef_,
index=X_train.columns, columns=['coef'])

print("\nCoefficients for logarithmic regression with transformed variables:")
print(coefficients_log_regression)
```

Coefficients for the logarithmic regression with transformed variables:

```
          coef
CRIM     -0.01
ZN        0.00
INDUS     0.00
CHAS      0.08
NOX      -0.70
RM        0.07
AGE       0.00
DIS      -0.05
RAD       0.01
TAX      -0.00
PTRATIO  -0.03
B         0.00
LSTAT    -0.03
```

For the most part, the coefficients retain their expected signs, which reinforces the model's reliability. For example, CHAS (proximity to the Charles River) has a positive coefficient, suggesting that properties near the river tend to command higher prices—likely due to the scenic beauty and desirability of the location.

On the other hand, the PTRATIO (students per teacher) has a negative coefficient, suggesting that areas with more crowded classrooms tend to have lower property values, which is consistent with the assumption that better school quality (smaller class sizes) leads to higher demand.

Overall, the logarithmically transformed model provides a more stable and realistic interpretation of how various factors influence property prices in relative terms.

Comparing Model Performance: Original vs. Log-Transformed Predictions

Building on our coefficient analysis, we will now examine how the log transformation affects the model's predictive performance. By comparing the actual and predicted values, as well as the residual patterns of both models, we can visually assess whether the transformation leads to a better fit.

This step helps us not only determine whether the model has improved numerically (as measured by the R^2 value), but also whether it behaves more consistently and realistically across different price ranges.

Structuring Visualization for Model Comparison

To efficiently compare multiple scenarios, we again define a configuration list that stores the display parameters for both the original model and the logarithmically transformed model.

This approach allows us to systematically create comparable visualizations without having to duplicate code, thereby ensuring the consistency of all charts.

```
plots_log = [
    {
        'x': y_train, 'y': predicted_values, 'color': '#7f3c8d',
        'title': f'Original: Actual vs Predicted (R² {r_squared:.3f})',
        'xlabel': 'Actual Prices ($1000s)', 'ylabel': 'Predicted Prices ($1000s)',
        'plot_type': 'regression'
    },
    {
        'x': log_y_train, 'y': log_predictions, 'color': '#11a579',
        'title': f'Log Transformed: Actual vs Predicted (R² {log_r_squared:.2f})',
        'xlabel': 'Actual Log Prices', 'ylabel': 'Predicted Log Prices',
        'plot_type': 'regression'
    },
    {
        'x': predicted_values, 'y': residuals, 'color': '#3969ac',
        'title': 'Original: Residuals vs Predicted',
        'xlabel': 'Predicted Prices ($1000s)', 'ylabel': 'Residuals',
        'plot_type': 'residual'
    },
    {
        'x': log_predictions, 'y': log_residuals, 'color': '#f2b701',
        'title': 'Log Transformed: Residuals vs Predicted',
        'xlabel': 'Predicted Log Prices', 'ylabel': 'Residuals',
        'plot_type': 'residual'
    }
]
```

Visualizing Predictions and Residuals Across Models

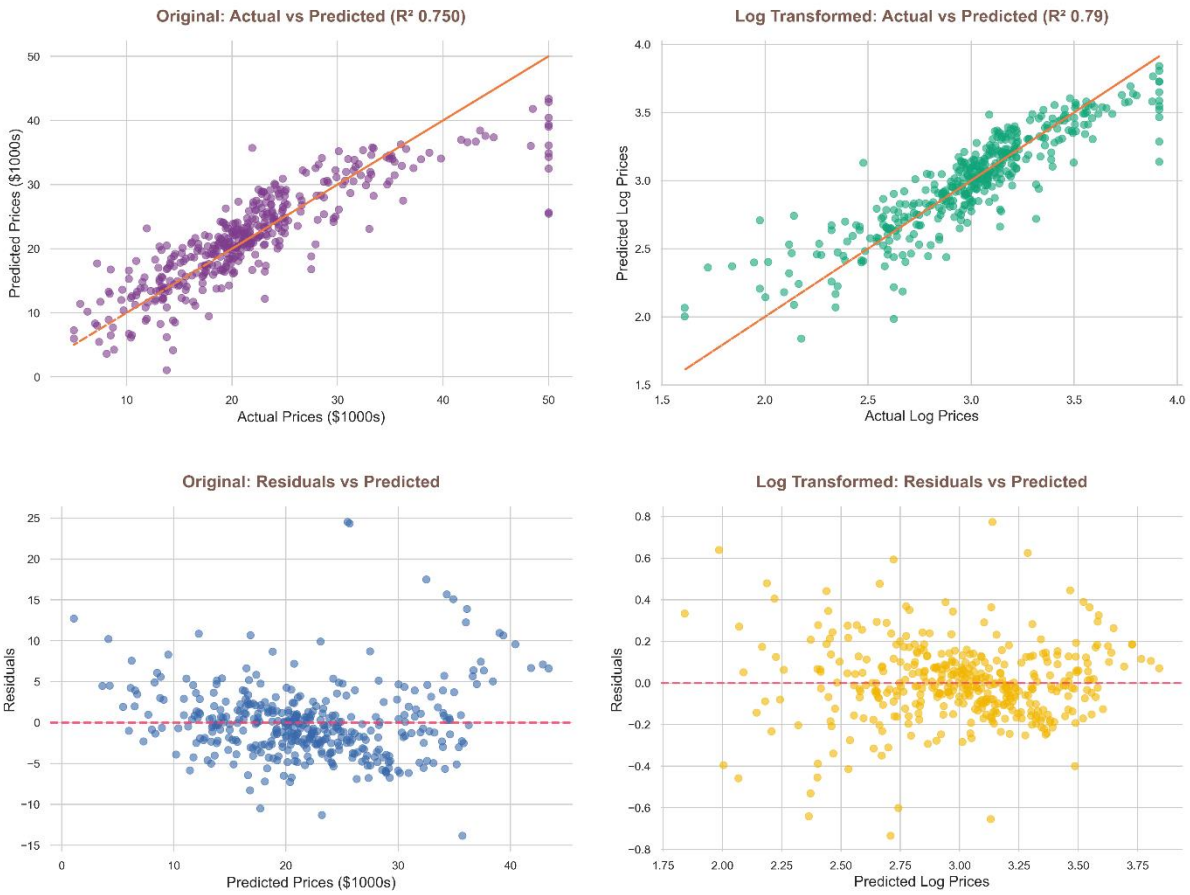
We create a 2×2 grid of scatter plots to compare the original and transformed models side by side.

This includes plots showing actual and target values to assess overall accuracy, as well as plots showing residuals and target values to evaluate the error distribution and model assumptions.

code:

```
sns.set_theme(style="whitegrid")

fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(16, 12), dpi=300)

# Looping through the plots and the configurations
# axes.flatten() turns the 2x2 grid into a simple 1D list so we can loop over it
easily
for ax, plot in zip(axes.flatten(), plots_log):
    sns.scatterplot(
        x=plot['x'], y=plot['y'],
        ax=ax, color=plot['color'],
        alpha=0.6, s=50, edgecolor=None
    )

    # axes-titles
    ax.set_title(plot['title'], fontsize=16, fontweight='bold', pad=18,
color='#7D5A50')
    ax.set_xlabel(plot['xlabel'], fontsize=14)
    ax.set_ylabel(plot['ylabel'], fontsize=14)

    # we need two types of lines here:
    if plot['plot_type'] == 'regression':
        ax.plot(plot['x'], plot['x'], color='#F07B3F', linestyle='--',
linewidth=2)
    elif plot['plot_type'] == 'residual':
        ax.axhline(y=0, color='#F6416C', linestyle='--', linewidth=2, alpha=0.8)


sns.despine(left=True, bottom=True)
plt.tight_layout(h_pad=4.0, w_pad=3.0)

plt.show()
```

Graphs:



The comparison clearly shows that the logarithmically transformed model provides a better fit, with predictions lying closer to the ideal line and an improved R^2 value (0.79 versus 0.75) indicating greater explanatory power.

In the residual plots, the log model exhibits a more uniform distribution around zero, whereas the original model shows increasing variance at higher price levels—an indication of heteroscedasticity, which is successfully reduced by the transformation.

This improvement is due to the fact that the log transformation compresses outliers, particularly for properties in the luxury segment, which are often influenced by complex factors such as the prestige of the location or unique features—making the data more consistent and easier for a linear model to capture accurately.

Comparing Residual Distributions: Original vs. Log-Transformed Model,

In this section, we are going to compare the residual distributions of the original model and the logarithmically transformed model to assess which model provides a better statistical fit.

By examining the mean and skewness of the residuals, we can assess how closely each model approximates the ideal case of a normal distribution with a mean of zero.

Based on this comparison, we can determine whether the log transformation has successfully reduced the bias error and improved the overall reliability of the model.

Distribution of Residuals (log prices):

```
log_residual_mean = round(log_residuals.mean(), 2)
log_residual_skew = round(log_residuals.skew(), 2)
```

Visualizing and Comparing Residual Distributions

To better understand the differences, we plot the residual distributions of both of the models side by side as histograms and KDE curves.

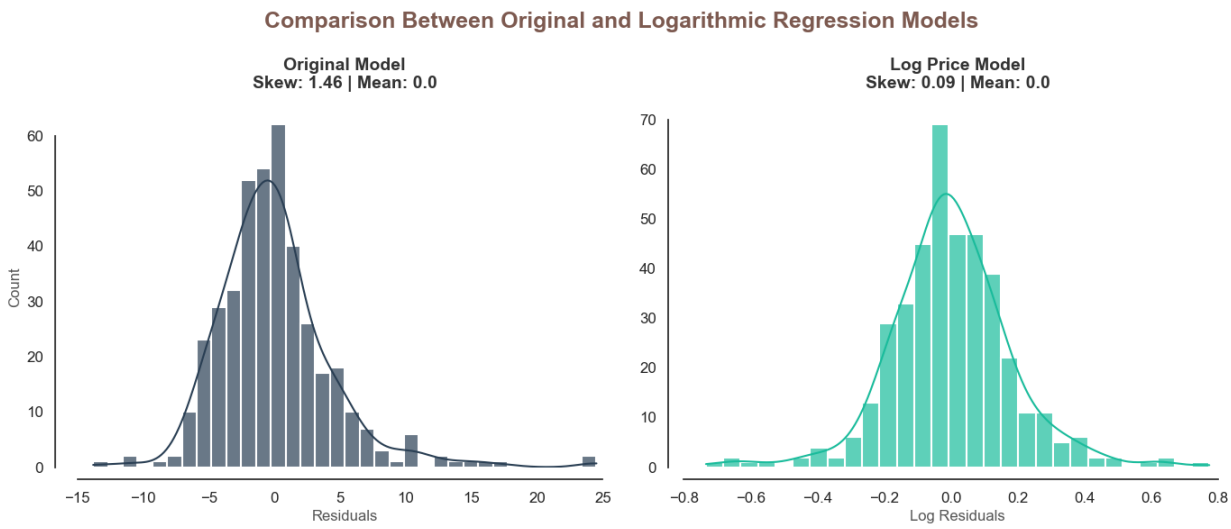
This allows us to visually assess the symmetry, dispersion, and presence of skewness in each case.

code:

```
sns.set_theme(style="white", font_scale=1.1)
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Original model plot:
sns.histplot(residuals, kde=True, color='#2A3F54', alpha=0.7,
             edgecolor="white", linewidth=1.5, ax=axes[0])

axes[0].set_title(f'Original Model\nSkew: {skew_residual} | Mean:
{mean_residual}',fontsize=14, weight='bold', color='#333333', pad=15)

axes[0].set_xlabel('Residuals', fontsize=12, color='#555555')
axes[0].set_ylabel('Count', fontsize=12, color='#555555')

# Log transformation plot:
sns.histplot(log_residuals, kde=True, color='#1ABC9C', alpha=0.7,
```

```
edgecolor="white", linewidth=1.5, ax=axes[1])

axes[1].set_title(f'Log Price Model\nSkew: {log_residual_skew} | Mean:
{log_residual_mean}',fontsize=14, weight='bold', color='#333333', pad=15)

axes[1].set_xlabel('Log Residuals', fontsize=12, color='#555555')
axes[1].set_ylabel('') # keep it empty for a cleaner look

fig.suptitle('Comparison Between Original and Logarithmic Regression Models',
fontsize=18, fontweight='bold', color='#7D5A50')

sns.despine(offset=10, trim=True)
plt.tight_layout()
plt.show()
```

Graph:



The comparison clearly shows that the logarithmically transformed model yields a significantly more symmetrical residual distribution, with the skewness decreasing from 1.46 to 0.09 and thus approaching the ideal value of zero.

This suggests that the transformed model has significantly reduced the bias error and handles outliers—particularly high-priced properties—more effectively than the original model. From both a residuals perspective and an r-squared perspective we have improved our model with the data transformation.

Evaluating Model Generalization on Unseen Test Data

In this section, we are going to use the test dataset to evaluate how well our models perform on new, previously unseen data.

While training performance provides insight into how well the model is tailored to known data, the true measure of success is the accuracy with which it can generalize to unseen inputs.

By comparing the R^2 values of both the original and the logarithmically transformed models using the test data, we can determine which model is more robust and reliable in real-world scenarios.

Simply put: The training data is used to teach the model patterns, while the test data serves as an objective benchmark for assessing how well these learned patterns can be applied to new information.

```
print(f'Coefficient of Distribution(r-squared) at Original Model Test Data:')
print(f"{Regression.score(X_test, y_test):.2}")
print(f'Coefficient of Distribution(r-squared) at Logarithmic Model Test Data:')
print(f"{log_regression.score(X_test, log_y_test):.2}")
```

```
Coefficient of Distribution(r-squared) at Original Model Test Data:
0.67
Coefficient of Distribution(r-squared) at Logarithmic Model Test Data:
0.74
```

The results show that the original model achieves an R^2 value of 0.67, while the logarithmically transformed model performs better on the test dataset, with an R^2 value of 0.74.

As expected, both values are slightly lower than the corresponding values from the training dataset, since the models were not directly optimized for this unknown data.

However, the relatively high R^2 values—particularly in the logarithmically transformed model—suggest that the model generalizes well and captures meaningful patterns rather than simply memorizing the training data.

This suggests that the log transformation not only improved training performance but also enhanced the model's ability to make more reliable predictions under real-world conditions.

Making Predictions: Estimating Property Value with the Model

In this section, we are going to use our trained regression model to estimate the value of a property based on its characteristics. By using the trained coefficients, we can input feature values and calculate a predicted price, which demonstrates the practical applicability of our model.

Since our preferred model uses a logarithmically transformed dependent variable, we first calculate the logarithmic value and then convert it back to a real dollar amount.

Constructing a Representative "Average" Property Profile

In this step, we create a baseline by calculating the mean values of all feature variables using the Pandas .mean() function. This is a common method in data science used to aggregate datasets in order to create a representative basis for analysis or predictions.

By converting these values into a one-row DataFrame, we ensure that the data structure matches the format required by our trained regression model.

```
features = data.drop(['PRICE'], axis=1)
average_property_features = features.mean().values

property_values = pd.DataFrame(data=average_property_features
                               .reshape(1, len(features.columns)),
                               columns=features.columns)

print("Average Property Features:")
print(property_values)
```

Average Property Features:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	3.61	11.36	11.14	0.07	0.55	6.28	68.57	3.80	9.55	408.24	18.46	356.67	12.65

These figures describe a "typical" property in Boston: for example, the average house has about 6.28 rooms, is located at a moderate distance (DIS ≈ 3.8) from workplaces, and is situated in areas with moderate air pollution (NOX ≈ 0.55).

Socioeconomic indicators such as LSTAT (~12.65%) and PTRATIO (~18.46) point to a well-balanced neighborhood with moderate income levels and average school conditions.

Overall, this profile provides a realistic benchmark that allows us to compare individual properties using a meaningful, data-driven standard.

Estimating Property Value Using the Trained Model:

In this step, we use our trained log-linear regression model to predict the value of the previously defined feature “average.”

The model first generates a prediction in logarithmic form, consistent with the way it was trained.

To make this result meaningful, we reverse the logarithmic transformation using the exponential function and convert the value back to actual dollar amounts and then multiply them by 1,000, as the prices are listed in thousands of dollars.

```
# Make prediction
log_estimate = log_regression.predict(property_values)[0]
# it returns a list with one value in it (price estimation)

# Convert Log Prices to Acutal Dollar Values
dollar_estimation = np.exp(log_estimate) * 1000

print(f'The property is estimated to be worth ${dollar_estimation:.6}')
```

```
The property is estimated to be worth $20703.2
```

The model estimates the average home value at approximately $20,703 in 1970 dollars. Adjusted for inflation (~740%), this amounts to roughly $173,906 in today's currency, which is in line with expectations for an average home.

This result suggests that our model provides realistic and consistent estimates, underscoring its usefulness for understanding and predicting real estate prices.

Scenario-Based Modeling: Estimating Price from Custom Property Preferences

In this section, we are going to go beyond average estimates and use a customized scenario to examine how specific property characteristics affect the price.

By adjusting selected factors such as number of rooms, pollution level, and proximity to the river, while keeping other factors constant, we can simulate how different conditions affect property value.

This approach demonstrates the practical effectiveness of our model, which enables us to estimate prices based on realistic buyer preferences and local conditions.

Defining Property Characteristics

Here, we adjust the key variables of our “average property” to create a more attractive scenario, for example by increasing the number of rooms and locating the property near a river.

By directly updating these attribute values in our dataset, we conduct a controlled experiment to isolate the impact of specific factors on property prices

```
next_to_river = True
nr_rooms = 8
students_per_classroom = 20
distance_to_town = 5
pollution = data.NOX.quantile(q=0.75) # high
amount_of_poverty =  data.LSTAT.quantile(q=0.25) # low
```

Applying Custom Property Features

```
# Set Property Characteristics
property_values['RM'] = nr_rooms
property_values['PTRATIO'] = students_per_classroom
property_values['DIS'] = distance_to_town
property_values['NOX'] = pollution
property_values['LSTAT'] = amount_of_poverty

if next_to_river:
    property_values['CHAS'] = 1
else:
    property_values['CHAS'] = 0
```

These are the values for a property that is completely average in terms of crime rate, year of construction, tax rate, and access to the highway network.

However, we are going to predict how the price changes when we adjust the number of rooms and the environmental impact and place the house next to a river.

Estimating the Price for the Customized Property:

We use the trained log-linear model to predict the logarithmic price of this custom-built property and then convert it back to its actual dollar value.

This allows us to interpret the result in realistic financial terms.

```
# Making the  prediction
log_estimate = log_regression.predict(property_values)[0]

# Convert Log Prices to Acutal Dollar Values
dollar_estimation = np.exp(log_estimate) * 1000

print(f'The value of the property that meets the desired criteria is estimated at ${dollar_estimation:.6}')
```

```
The value of the property that meets the desired criteria is estimated at $25792.0
```

The model estimates that this property would have been worth approximately $25,792 in 1970.

Adjusted for inflation (~740%), this corresponds to approximately $216,652 today, suggesting that features such as larger living space, a lower poverty rate, and a location at the side of the river significantly increase property value, even when other factors are average.

Optimizing Property Value: Finding the Best House Within Budget Constraints

In this section, we are going to take our analysis a step further by formulating the problem as an optimization task:

The goal is to find the most suitable property that meets specific preferences while staying within a set budget. Instead of simply predicting prices, we now want to reverse the problem: Given the desired characteristics, what is the best possible house we can purchase at the lowest price?

Let's imagine David Carter, a civil engineer in middle management who works in Boston, and his wife Emily Carter, a public school teacher. They have two children, ages 8 and 11, and are looking for a safe, family-friendly neighborhood where they can settle down. David prefers a home close to nature—especially near the Charles River—while Emily values good schools, which is why a low student-teacher ratio is important to her. They are also looking for a spacious home (6–8 rooms) in an area with low crime and poverty rates, preferably with minimal environmental impact and not too far from their workplaces; interestingly, however, they place less importance on our data points, such as the full property tax rate per household or the density of the African American population in the region.

Like most families, however, they have to make do with limited financial resources—their maximum budget is $30,000 (as of 1970). Instead of searching at random, we use data science methods to identify the optimal combination of features that match their preferences while minimizing costs.

For features that we do not consider particularly important, we keep the values within a reasonable range to allow the model flexibility. This transforms the problem into a structured mathematical optimization problem, in which we balance multiple variables against one another to find the most cost-effective solution.

Here is a general value scales for the important and decisive parameters that we define for the Optimization methodology.

```
Crime Rate (CRIM): (0.0, 1.4)
Reserved-Land percentage (INDUS): (5.0, 12.0)
River (CHAS): (1, 1)
Nitrogen dioxide concentration (NOX): (0.0, 0.43)
Room Number (RM): (6.0, 8.0)
Distance to work (RM): (1.0, 3.0)
Accessibility to Roads(RAD): (5.0, 18.0)
```

```
Student-teacher ratio(PTRATIO): (12.6, 15.0)
Lower status population(LSTAT): (1.0, 9.0)
```

Applying Optimization Techniques

```
from scipy.optimize import minimize
```

To solve this problem, we use the $.minimize()$ function from SciPy, a powerful tool for solving optimization problems with constraints. We define an objective function that minimizes the predicted (logarithmic) price and apply a budget constraint to ensure that the final price does not exceed $30,000. By setting thresholds for each attribute, we guide the optimizer to search within realistic and meaningful ranges while taking the buyer's preferences into account.

Finding the Optimal Property Configuration

The optimizer adjusts the attribute values incrementally to identify the combination that achieves the lowest possible price while satisfying all constraints.

This approach utilizes both machine learning (regression model) and mathematical optimization, making it a practical and advanced application of data science.

The final result provides not only the estimated price of the ideal property, but also the specific property characteristics that lead to that price.

This provides us with valuable insights into trade-offs—for example, regarding the extent to which room size, location, or school quality can be adjusted to stay within budget, making the analysis extremely useful for practical decision-making.

```
#  Defining the Objective Function (Minimize Price)
def objective(features):
    # The optimizer continuously passes new feature arrays to this function.
    # We want to minimize the predicted logarithm of the price.
    # lowering the logarithm of the price also lowers the actual price
    predicted_log_price = log_regression.predict([features])[0]
    return predicted_log_price
```

```python
# Defining the Constraints (Max Budget: 30.0)
# SciPy constraints work by checking if the result is >= 0.
# So: np.log(30.0) - predicted_log_price >= 0
def budget_constraint(features):
    target_log_price = np.log(30.0)
    current_log_prediction = log_regression.predict([features])[0]
    return target_log_price - current_log_prediction


# Pack the constraint into a dictionary for SciPy
constraints = [{'type': 'ineq', 'fun': budget_constraint}]

# Defining the Bounds (Desired Features)
# We are going to provide a (min, max) for all 13 features so the optimizer
bounds = [
    (0.0, 1.4),  # CRIM (Crime rate):
    (0.0, 15.0),  # ZN # unimportant
    (5.0, 12.0),  # INDUS
    (1, 1),  # CHAS (0 or 1 for River)
    (0.0, 0.43),  # NOX (Pollution)
    (6.0, 8.0),  # RM (Rooms)
    (40.0, 80.0),  # AGE
    (1.0, 3.0),  # DIS (Distance to work)
    (5.0, 18.0),  # RAD
    (300.0, 600.0),  # TAX unimportant, broad scale
    (12.6, 15.0),  # PTRATIO (Student-teacher ratio)
    (356.0, 356.0),  # B # unimportant, mean value
    (1.0, 9.0)  # LSTAT (Lower status population %)
]


# Setting Initial Guess and Run Optimizer
initial_guess = average_property_features.copy()

result = minimize(
    fun=objective,
    x0=initial_guess,
    method='SLSQP',
    bounds=bounds,
    constraints=constraints
)


if result.success:
    print("Optimization Successful!\n")

    # Calculating final actual price (converting log to normal)
    optimal_house_log_price = result.fun
    optimal_house_actual_price = np.exp(optimal_house_log_price) * 1000
    print(f"Optimized Price for the Desired House:
```

```
${optimal_house_actual_price:,.2f}\n")

    print("All of the Conditions of Ideal House:")
    optimal_features = result.x
    for name, value in zip(features.columns, optimal_features):
        print(f"{name:>8}: {value:.2f}")
else:
    print("Optimization failed to find a solution. Theconstraints might be too
strict.")
```

The output:

Optimization Successful!

Optimized Price for the Desired House: $28,049.15

All of the Conditions of Ideal House:

```
    CRIM: 1.40
      ZN: 0.00
   INDUS: 5.00
    CHAS: 1.00
     NOX: 0.43
      RM: 6.00
     AGE: 40.00
     DIS: 3.00
     RAD: 5.00
     TAX: 432.38
 PTRATIO: 15.00
       B: 356.00
   LSTAT: 9.00
```

The optimization results suggest that David and Emily Carter can find a house for about $28,049 (1970 value) in Boston that meets almost all of their criteria; adjusted for inflation, this amounts to nearly $235,611 today.

The model balances her preferences by selecting a moderate crime rate, a riverside location, and acceptable school quality (PTRATIO ≈ 15), while making slight

compromises on factors such as pollution and the number of rooms (at the lower limit of 6) in order to stay within budget.

This result reflects a realistic compromise: even if you have high standards regarding living space, location, and the quality of the neighborhood, meeting all these requirements at a lower cost requires a certain degree of flexibility in less critical areas.

For the Carter family, this means securing a safe, well-located home near the river with adequate educational opportunities, while accepting a slightly higher environmental impact and average infrastructure conditions.

This also demonstrates how, even in complex real estate markets, a structured analysis can transform subjective preferences into clear, actionable decisions.

Overall, this finding demonstrates how data-driven optimization can help families make smart and balanced decisions about their housing situation that align with both their life goals and their financial circumstances.

Conclusion:

In this report, we examined the real estate market in Boston during the 1970s to understand what factors determine real estate prices and how various factors affect property values. Using a structured analysis, we examined the relationships between environmental, structural, and socioeconomic variables and ultimately developed models that can estimate real estate prices with reasonable accuracy. By the end of the analysis, we were able not only to identify the key factors that influence property values such as apartment size, environmental pollution, and socioeconomic conditions—but also to demonstrate how these factors interact in realistic scenarios, including an optimized property selection based on preferences and budget.

From a data science perspective, this project encompassed the entire analytical workflow—from data cleaning and validation through exploratory data analysis to visualization and predictive modeling. We used tools such as Pandas for data processing, Seaborn and Matplotlib for customized visualizations, and Scikit-learn for regression models, carefully evaluating both the statistical results and the visual patterns. To improve the model's performance, advanced techniques such as log transformation were applied, while the model's reliability was ensured through residual analysis and performance metrics (R^2). Finally, we expanded our analysis using optimization methods (SciPy) to simulate real-world decision-making processes, thereby demonstrating how data can be transformed into actionable insights as an indispensable skill in the practical application of data science.

References:

https://www.udemy.com/course/100-days-of-code/

https://pandas.pydata.org/docs/user_guide/index.html#user-guide

https://matplotlib.org/stable/users/index.html

https://seaborn.pydata.org/generated/seaborn.histplot.html

https://docs.scipy.org/doc/scipy/tutorial/index.html#user-guide

https://scikit-learn.org/stable/getting_started.html

https://docs.scipy.org/doc/scipy/reference/optimize.html

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

https://scikit-learn.org/stable/modules/tree.html#regression